

VIT BHOPAL UNIVERSITY

B.Tech Computer Science and Engineering (AI/ML specialization)
First Year, First Semester

Scam URL Checker

A Python program that checks a link and gives it a scam risk score

PROJECT REPORT

Submitted by: **Harshit Saraswat**

Registration No.: **26BAI10038**

Academic year 2026-27

Index of Figures and Tables

Item	Title	Section
Figure 1	System architecture of the Scam URL Checker	3
Table 1	Project files and what each one does	3
Table 2	Scoring rules used by the program	6.4
Table 3	Risk levels and the message shown for each	7.3
Table 4	Test cases and their results	8

Table of Contents

Index of Figures and Tables	2
Table of Contents	3
1. Introduction	4
2. Problem Statement and Objectives	4
2.1 Problem Statement	4
2.2 Objectives	4
3. System Architecture	5
4. Technology Justification	6
5. Data Structures and Data Models	6
5.1 The Blacklist	6
5.2 Values Returned by the Checkers	7
5.3 Other Data Used by the Program	7
6. Detailed Module Specification	7
6.1 main.py	7
6.2 domainextractor.py	7
6.3 blacklist.py	8
6.4 patternchecker.py	8
7. Implementation	8
7.1 Getting the Domain Out of the URL	8
7.2 Adding Up the Score	9
7.3 The Risk Report and Risk Levels	9
8. Testing and Verification	9
9. Conclusion	11

1. Introduction

Nowadays many people get messages like "your bank account will be blocked, click here" or "you won a free iPhone". The link in the message usually opens a fake website that looks like PayPal, Amazon or SBI. It is made to steal passwords or card numbers. If you look at the link carefully it is easy to spot, but most people click in a hurry.

This project is a small Python program that checks a link and gives it a risk score from 0 to 100. I made it in my first semester using what we learned in class: functions, loops, if/else, strings and using more than one file. It does not use machine learning and it does not need internet. It only uses simple rules, and it shows the reason for every point added to the score.

It is not a replacement for antivirus or the safe browsing in Chrome. It only takes the things a person checks by eye, like the domain name, https and words like "verify" or "secure", and does them using code. This report explains how the program is made, how I tested it and what its problems are.

Section 2 gives the problem statement and objectives. Sections 3 and 4 explain the structure and the tools used. Section 5 is about the data, Section 6 is about each file and Section 7 is about the implementation. Section 8 has the testing and Section 9 is the conclusion.

2. Problem Statement and Objectives

2.1 Problem Statement

Scam links copy the names of well known companies. They change a letter here and there (paypa1, amaz0n, g00gle), add words such as "verify", "secure" or "update", and stack up hyphens to make a long domain look official. A normal user has to judge all this just by looking at the link, which is difficult on a phone screen, especially when the message says the account will be closed in an hour.

The problem this project tries to solve is: **build a program that takes a URL from the user, checks it for common warning signs, and tells the user how risky it looks, along with the reasons**, instead of only saying "safe" or "not safe".

2.2 Objectives

- Take a URL as input and reject anything that does not start with http:// or https://.
- Pull the domain name out of the URL.
- Compare the domain with a stored list of known scam domains.
- Look for warning patterns: too many hyphens, a very short domain, no https, suspicious words, and a domain that looks like an IP address.

- Add the results into one score between 0 and 100 and turn it into a risk level with a short advice line.
- Show a list of flags so the user can see why the score came out that way.
- Keep the code in separate files and use no external libraries, so it runs anywhere with Python 3.6 or newer.

Anything beyond this, like checking a real SSL certificate, opening the site, or learning from data, is outside the scope of this version. Those points are listed in the conclusion as future work.

3. System Architecture

The program is a simple pipeline. The URL goes in at the top, each file does one job on it, and the report comes out at the bottom. Figure 1 shows the flow and Table 1 lists the files.

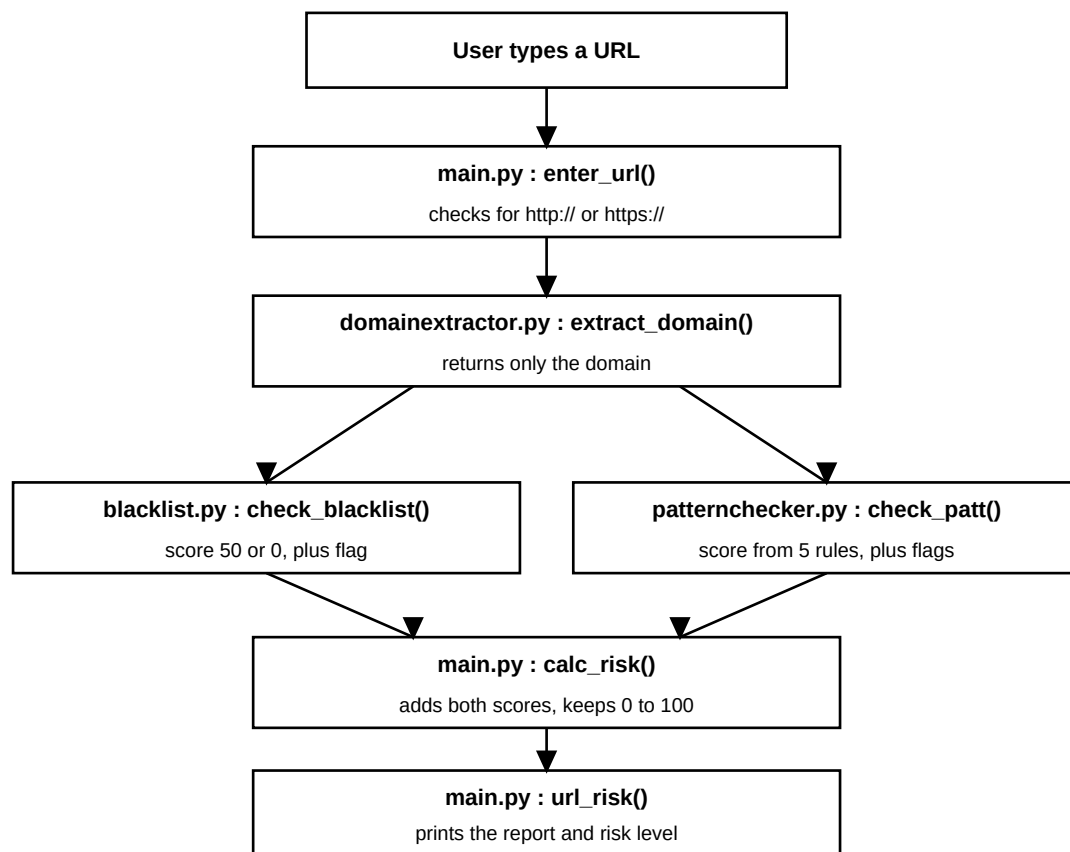


Figure 1: System architecture of the Scam URL Checker

File	What it does	Functions
main.py	Takes the input, calls the other three files, adds up the score and prints the report	enter_url(), calc_risk(), url_risk(), work()
domainextractor.py	Cuts the domain name out of the full URL	extract_domain()
blacklist.py	Holds the list of scam domains and checks the domain against it	check_blacklist()

File	What it does	Functions
patternchecker.py	Looks for suspicious patterns in the domain and the URL	check_patt()

Table 1: Project files and what each one does

Only main.py is run by the user. The other three files are imported by it. Because each file has one job, changing one part (for example, adding a new rule to the pattern checker) does not need any change in the others. The two checkers work independently: the blacklist file only sees the domain, and the pattern file sees both the domain and the full URL. Both return a score and a list of flags, and main.py combines them.

4. Technology Justification

Python: we are learning it in class, and lists and strings are already built in, which is what this project needs. There is no compile step so I could test changes quickly.

No external libraries: the program runs on a normal Python install and nothing has to be installed. Python has `urllib.parse` which can get the domain in one line, but I wrote the extraction myself to practice loops and string slicing.

Rules instead of machine learning: a model needs a lot of good and bad links to learn from, and I did not have that. It would also be hard to explain its answer. With rules, every point in the score comes from a line of code and the flags tell the user what was found.

List for the blacklist: with around 150 entries a list in the code is the easiest way. A database would be better for a big list that changes often, but it is too much for this project. The problem is that the list stays the same unless someone edits the code.

Terminal: it is the fastest to build and test. A GUI can be added later.

Separate files: each file has one job, so I could write and test one part at a time.

5. Data Structures and Data Models

This project does not use a database. All the data lives in ordinary Python variables, so this section describes those instead of a database schema.

5.1 The Blacklist

The blacklist is a Python list of strings, kept inside `check_blacklist()`. It holds around 150 domain names. Each entry is a full domain in lowercase, such as `paypal-login.com`. The domains are grouped by the kind of scam they copy:

- fake PayPal, Amazon, Google, Facebook, Instagram, WhatsApp, Apple and Microsoft pages
- fake Netflix, Spotify, Disney Plus and Hulu billing pages

- fake bank pages copying SBI, HDFC, ICICI, Axis, Chase, Wells Fargo, Barclays, HSBC and Bank of America
- fake tax and government pages (IRS, HMRC, Aadhaar, PAN card, passport)
- lottery, free gift and prize scams
- crypto scams (Binance, Coinbase, MetaMask, giveaways and airdrops)
- general "verify your account" and "password expired" type domains
- names that mix in URL shortener words or random letters and numbers
- delivery, refund and subscription scams

5.2 Values Returned by the Checkers

Both checkers return a pair: a whole number score and a list of text flags. The flag list is what makes the final report explain itself.

```
check_blacklist("paypal-login.com")    returns (50, ["Sir domain found in the scam_database"])
check_blacklist("google.com")          returns (0, [])
```

5.3 Other Data Used by the Program

- **Suspicious words:** a list of eight words - login, verify, confirm, update, secure, account, suspend, alert.
- **Delimiters:** a list of four characters (/ ? : #) that mark where the domain ends in a URL.
- **Final score:** a whole number kept between 0 and 100.
- **Risk levels:** fixed score ranges set with if/elif conditions, described in Section 7.3.

6. Detailed Module Specification

6.1 main.py

This is the file the user runs. It has four functions.

- `enter_url()` asks the user for a URL and checks that the text contains `http://` or `https://`. If not, it prints an error and returns `None`, and the program stops.
- `calc_risk(blacklist_score, patt_score)` adds the two scores. If the total is below 0 it becomes 0, and if it is above 100 it becomes 100.
- `url_risk(domain, score, flag)` prints the report: a heading box, the domain, the score, the list of flags, and the risk level with a line of advice.
- `work()` runs everything in order: input, domain extraction, blacklist check, pattern check, score, report. It is started by the usual `if __name__ == "__main__"` check at the bottom.

6.2 domainextractor.py

It has one function, `extract_domain(url)`. It takes the URL as text and returns just the domain. It removes spaces, then the `http://` or `https://` at the start, then `www.` if present, and finally copies characters one by one until it meets one of `/ ? : #`. Because the colon is a stopping point, a port number such as `:8080` is dropped as well.

6.3 blacklist.py

It has one function, `check_blacklist(domain)`. It turns the domain into lowercase and loops through the list. If the domain is exactly equal to an entry, it prints a critical message, adds a flag and returns 50 as the score. Otherwise it returns 0 and an empty list. The match has to be exact, so the check does not catch a subdomain like `login.paypal-login.com`.

6.4 patternchecker.py

It has one function, `check_patt(domain, url)`. It goes through five rules and adds points for every rule that applies. Table 2 lists them along with the blacklist rule for comparison.

Rule	Condition in the code	Points
Blacklist match	domain (in lowercase) equals an entry in the list	50
Too many hyphens	more than 2 hyphens in the domain	10
Short domain	fewer than 5 characters	5
No https	the text "https://" is not in the URL	15
Suspicious word	one of the eight words appears anywhere in the URL (counted once)	8
IP-like domain	exactly 3 dots in the domain	20

Table 2: Scoring rules used by the program

The keyword loop stops at the first word it finds, using `break`, so a URL with many of these words still gets only 8 points.

7. Implementation

7.1 Getting the Domain Out of the URL

A URL can contain a path, a query and a fragment after the domain, and none of that is needed for the checks. The function removes the front part first and then reads characters until it reaches a delimiter. For example, `https://www.paypal-login.com/verify?id=1` becomes `paypal-login.com`. This is the loop that does the copying:

```
delimiters = ["/", "?", ":", "#"]
domain = ""

for ch in clean_url:
    if ch in delimiters:
```

```
        break
    domain = domain + ch
```

It is simple, but it does have limits. A link with a username in it, like `http://user@site.com`, would not be cut correctly, and I did not handle that case.

7.2 Adding Up the Score

Each checker returns its own score, and `calc_risk()` adds them. Points from different rules can pile up past 100 in theory, so the function caps the total. The check for a negative total is there for safety, though none of the current rules can subtract points.

```
def calc_risk(blacklist_score, patt_score):
    t_score = blacklist_score + patt_score

    if t_score < 0:
        t_score = 0
    if t_score > 100:
        t_score = 100

    return t_score
```

7.3 The Risk Report and Risk Levels

`url_risk()` prints a heading box built with the `center()` string method, then the domain, the score, and every flag on its own line. After that it picks a risk level with an `if/elif` chain. Table 3 shows the ranges I intended and what the program prints.

Score	Level shown	Advice printed
80 to 100	Critical	do not visit this site
65 to 79	High	very suspicious, avoid if possible
40 to 64	Mid	a bit suspicious, proceed with caution
15 to 30	Kind of low	kind of suspicious, be a bit alert
0 to 14 and 31 to 39	Very low	likely safe, no need to worry

Table 3: Risk levels and the message shown for each

The last row needs an explanation. There is no condition written for scores from 31 to 39, so those scores fall through to the final `else` branch and are shown as "Very low", which is wrong. I found this only after the program was finished. Test case 11 in Section 8 shows it happening. The fix is to write the conditions as a clean chain, for example `score >= 80`, then `score >= 65`, then `score >= 40`, and so on, so that every number lands in exactly one branch.

8. Testing and Verification

I tested the program by running `main.py` with different URLs. For each one I worked out the expected score by hand from Table 2 and then compared it with what the program printed.

All twelve cases matched my hand calculation. Some of them show weaknesses in the rules and those are marked in the comment column. Screenshots of the runs are kept in the `screenshots` folder of the repository.

No.	URL entered	How the score adds up	Score	Level shown	Comment
1	https://google.com	no rule applies	0	Very low	normal site, no flags
2	https://paypal-login.com	blacklist 50 + word 8	58	Mid	caught by the list
3	https://amazon-account-verify.info	blacklist 50 + word 8	58	Mid	caught by the list
4	http://suspicious-site.com/login	no https 15 + word 8	23	Kind of low	not on the list
5	http://192.168.1.1/login	no https 15 + word 8 + IP 20	43	Mid	IP address detected
6	https://xk92jf-secure-login-verify-paypal.com	blacklist 50 + hyphens 10 + word 8	68	High	highest score in these tests
7	http://a.co	short 5 + no https 15	20	Kind of low	short domain flagged
8	ftp://example.com	rejected at input	-	-	error message shown
9	(only pressed Enter)	rejected at input	-	-	error shown, but the wrong one (see below)
10	https://secure-login-verify-account.com	hyphens 10 + word 8	18	Kind of low	looks like phishing but scores low
11	http://login-secure-verify-update.com/account	hyphens 10 + no https 15 + word 8	33	Very low	wrong level, the 31 to 39 gap
12	https://mail.example.co.uk	IP rule 20	20	Kind of low	false alarm, only counts dots

Table 4: Test cases and their results

What the tests showed. The normal cases work as planned. Known scam domains and links with several warning signs get a Mid or High score, and a clean link gets 0. The problems are these:

- **The 31 to 39 gap** (case 11). A link with three warning signs is shown as "Very low" because of the missing condition.
- **Unlisted phishing links score low** (case 10). A fake login page on a new domain that uses https and has only a couple of hyphens gets very few points from the pattern rules, so it looks almost safe.
- **The IP check counts dots only** (case 12). Any domain with exactly three dots, like `mail.example.co.uk`, is flagged as an IP address.
- **The empty input message is never reached** (case 9). In `enter_url()` the http/https check comes first, and an empty string fails it, so the user sees the "must start with http"

error and the "can't be empty" check below it never runs.

- **Keywords are searched in the whole URL**, not just the domain, so a harmless page like `https://example.com/account` also gets 8 points.
- **Exact matching in the blacklist** means a subdomain of a listed domain, or a slightly changed spelling, is not caught.

I did not change the code after finding these, so the report matches the code that is uploaded. They are the first things to fix in the next version.

9. Conclusion

The project does what I wanted. It takes a URL, gets the domain, checks it against a list of scam domains, checks five warning patterns, and shows a score out of 100 with the reasons and a risk level. It works well for links that are on the blacklist or have many warning signs together, and a clean link gets 0.

It also has problems, and testing showed them. The blacklist is fixed inside the code and needs an exact match. The pattern rules are simple, so a fake page on a new domain can get a low score, and a normal domain can be flagged by mistake. There is a gap in the risk levels between 31 and 39. It does not check a real SSL certificate and it does not use any learning model.

Future work: first fix the bugs from Section 8. After that I want to add a GUI, load the scam list from the internet so it stays updated, check the real SSL certificate, compare domains with brand names to catch look-alike spellings, and maybe make a web app or browser extension.

What I learned: I practiced writing functions and passing values between them, using loops on lists and strings, slicing and checking a string inside another string, and dividing a program into files. The biggest lesson was testing. I found the 31 to 39 gap and the empty input problem only after running many different inputs.